

Curso desarrollo de aplicaciones con tecnologías web

UF1842. Desarrollo y reutilización de componentes software y multimedia mediante lenguajes de gui3n.

Contenidos multimedia

Animaciones multimedia

Elementos Interactivos

Animaciones multimedia

Principios de la Animación

El vídeo es un componente fundamental en la experiencia digital actual. Desde contenido educativo hasta publicidad interactiva, la correcta integración de vídeo en la web permite mejorar la experiencia del usuario y optimizar la carga y visualización de los recursos multimedia.

La animación en la web se basa en los mismos principios que en el cine y la animación tradicional:

- **Anticipación:** Preparar al usuario para una acción.
- **Aceleración y desaceleración:** Movimiento fluido y natural.
- **Arcos de movimiento:** Trayectorias curvas en lugar de líneas rectas.
- **Exageración:** Resaltar cambios para mejorar la percepción.
- **Timing y espaciado:** Control de la velocidad y fluidez de la animación.

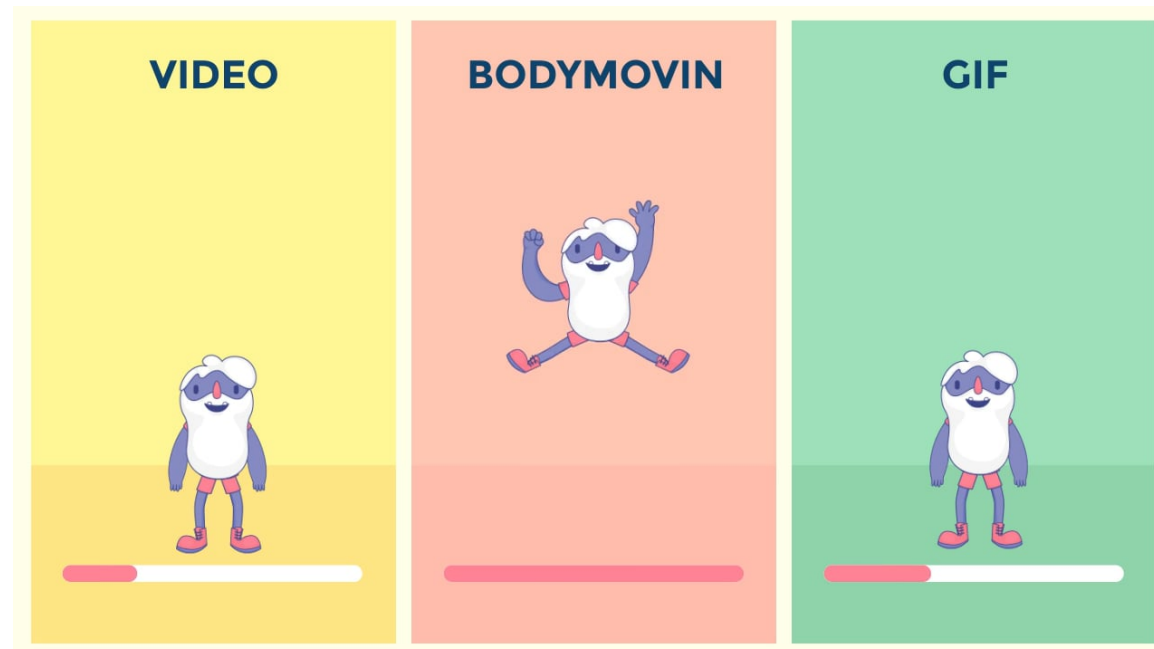
Herramientas para la Creación de Animaciones

- **CSS (Transitions & Animations):** Para animaciones simples y eficientes.
- **JavaScript (GSAP, Anime.js):** Mayor control y dinamismo.
- **SVG + CSS/JS:** Para animaciones vectoriales escalables.
- **WebGL & Three.js:** Para gráficos avanzados en 3D.
- **Canvas API:** Para animaciones interactivas mediante JavaScript.

Formatos de Animaciones

- **GIF:** Limitado en calidad y colores.
- **APNG:** Alternativa moderna al GIF.
- **SVG:** Animación basada en vectores.
- **CSS & JavaScript:** Para animaciones dinámicas.
- **WebM & MP4:** Animaciones en formato de vídeo optimizado.
- **Lottie (JSON + Bodymovin):** Para animaciones exportadas desde Adobe After Effects.

<https://lottiefiles.com/>



Inclusión en Páginas Web

- Ejemplo con CSS:

```
<div class="animated-box"></div>
<style>
  .animated-box {
    width: 100px;
    height: 100px;
    background-color: red;
    animation: move 2s infinite alternate;
  }

  @keyframes move {
    from {
      transform: translateX(0);
    }

    to {
      transform: translateX(100px);
    }
  }
</style>
```

- Ejemplo con JavaScript:

```
<button id="animate">Animar</button>
<div id="box"></div>
<script>
  document.getElementById("animate").addEventListener("click", () => {
    document.getElementById("box").classList.toggle("move");
  });
</script>
<style>
  #box {
    width: 100px;
    height: 100px;
    background: blue;
  }

  .move {
    transform: translateX(200px);
    transition: transform 1s;
  }
</style>
```

- Canvas en HTML5

El elemento <canvas> permite dibujar gráficos y animaciones de manera programada usando **JavaScript**. Se utiliza en lugar de tecnologías obsoletas como Flash para crear juegos, gráficos interactivos, visualizaciones de datos, etc.

html

```
<canvas id="myCanvas" width="500" height="300"></canvas>
```

Para dibujar en el canvas, necesitamos un contexto 2D:

js:

```
const canvas = document.getElementById("myCanvas");  
const ctx = canvas.getContext("2d");
```

Ahora podemos dibujar formas básicas:

js:

```
// Rectángulo  
ctx.fillStyle = "blue";  
ctx.fillRect(50, 50, 100, 100);  
  
// Círculo  
ctx.beginPath();  
ctx.arc(250, 150, 50, 0, Math.PI * 2);  
ctx.fillStyle = "red";  
ctx.fill();  
ctx.closePath();
```

- **Mini juego tipo “Pong” con Canvas**

Vamos a desarrollar una versión sencilla del clásico Pong usando Canvas y JavaScript.



Características del juego

- Pelota en movimiento.
- Raqueta controlada por el jugador.
- Rebote contra los bordes.
- Reinicio si la pelota sale por abajo.



Explicación

1. Se obtiene el lienzo (canvas) y su contexto 2d, que permite dibujar en él.

```
const canvas = document.getElementById("gameCanvas");  
const ctx = canvas.getContext("2d");
```

2. Se define su posición inicial en el centro de la pantalla y su velocidad en x e y. También se define el radio de la pelota.

```
// Configuración de la pelota  
let ballX = canvas.width / 2;  
let ballY = canvas.height - 30;  
let ballDX = 2;  
let ballDY = -2;  
const ballRadius = 10;
```


3. Configuración de la raqueta

Se establece el tamaño de la raqueta y su posición inicial en la parte inferior del canvas.

```
// Configuración de la raqueta
const paddleHeight = 10;
const paddleWidth = 75;
let paddleX = (canvas.width - paddleWidth) / 2;
```

Se crean variables para detectar si las teclas están siendo presionadas.

```
let rightPressed = false;
let leftPressed = false;
```

4. Controles del teclado

Se detecta cuando el usuario presiona o suelta las teclas de flecha izquierda y derecha. Actualización: Se usa `requestAnimationFrame(updateGame)`; para refrescar la animación continuamente.

```
// Controles
document.addEventListener("keydown", keyDownHandler);
document.addEventListener("keyup", keyUpHandler);

function keyDownHandler(e) {
  if (e.key === "Right" || e.key === "ArrowRight") rightPressed = true;
  else if (e.key === "Left" || e.key === "ArrowLeft") leftPressed = true;
}

function keyUpHandler(e) {
  if (e.key === "Right" || e.key === "ArrowRight") rightPressed = false;
  else if (e.key === "Left" || e.key === "ArrowLeft") leftPressed = false;
}
```

5. Dibujar la pelota

Se usa `ctx.arc()` para dibujar un círculo amarillo en la posición `ballX`, `ballY`.

```
// Dibujar la pelota
function drawBall() {
  ctx.beginPath();
  ctx.arc(ballX, ballY, ballRadius, 0, Math.PI * 2);
  ctx.fillStyle = "yellow";
  ctx.fill();
  ctx.closePath();
}
```

6. Dibujar la raqueta

Se usa `ctx.fillRect()` para dibujar un rectángulo azul en la posición `paddleX`.

```
// Dibujar la raqueta
function drawPaddle() {
  ctx.fillStyle = "blue";
  ctx.fillRect(paddleX, canvas.height - paddleHeight, paddleWidth, paddleHeight);
}
```

7. Lógica de colisiones

Aquí es donde se maneja la lógica de rebotes:

- Rebote en los bordes laterales

Si la pelota toca el borde derecho o izquierdo, cambia la dirección horizontal (`ballDX`).

```
// Rebote en los laterales
if (ballX + ballDX > canvas.width - ballRadius || ballX + ballDX < ballRadius) {
  ballDX = -ballDX;
}
```

- Rebote en la parte superior

Si la pelota toca el borde superior, cambia la dirección vertical (ballDY).

```
// Rebote en la parte superior
if (ballY + ballDY < ballRadius) {
    ballDY = -ballDY;
}
```

- Colisión con la raqueta

Si la pelota alcanza la parte inferior del canvas, se verifica si golpea la raqueta.

```
// Colisión con la raqueta
else if (ballY + ballDY > canvas.height - paddleHeight) {
    if (ballX > paddleX && ballX < paddleX + paddleWidth) {
        ballDY = -ballDY;
    } else {
        // Fin del juego
        document.location.reload();
    }
}
```

8. Movimiento de la raqueta

Se mueve la raqueta a la derecha o a la izquierda dependiendo de las teclas presionadas.

```
// Movimiento de la raqueta
if (rightPressed && paddleX < canvas.width - paddleWidth) {
    paddleX += 7;
} else if (leftPressed && paddleX > 0) {
    paddleX -= 7;
}
```

9. Movimiento de la pelota

La pelota se mueve sumando ballDX y ballDY a su posición actual.

```
// Movimiento de la pelota
ballX += ballDX;
ballY += ballDY;
```

10. Animación continua

requestAnimationFrame(updateGame) asegura que el juego se actualice continuamente.

```
// Movimiento y colisiones
function updateGame() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  drawBall();
  drawPaddle();

  // Rebote en los laterales
  if (ballX + ballDX > canvas.width - ballRadius || ballX + ballDX < ballRadius) {
    ballDX = -ballDX;
  }

  // Rebote en la parte superior
  if (ballY + ballDY < ballRadius) {
    ballDY = -ballDY;
  }

  // Colisión con la raqueta
  else if (ballY + ballDY > canvas.height - paddleHeight) {
    if (ballX > paddleX && ballX < paddleX + paddleWidth) {
      ballDY = -ballDY;
    } else {
      // Fin del juego
      document.location.reload();
    }
  }
}
```

```
// Movimiento de la raqueta
if (rightPressed && paddleX < canvas.width - paddleWidth) {
    paddleX += 7;
} else if (leftPressed && paddleX > 0) {
    paddleX -= 7;
}

// Movimiento de la pelota
ballX += ballDX;
ballY += ballDY;

requestAnimationFrame(updateGame);
}

updateGame();
```

Este es un ejemplo básico que se puede expandir agregando puntuación, niveles y efectos visuales.

- **Ejemplo de Usos del <canvas> en Aplicaciones Reales**

El elemento <canvas> de HTML5 es muy potente y se usa en diferentes ámbitos más allá de simples dibujos. Aquí hay algunos **casos de uso reales**:

1. **Juegos y Animaciones Interactivas con Adobe Animate**

El sucesor de Flash, Adobe Animate, permite exportar animaciones y juegos en formato HTML5 Canvas + JavaScript en lugar de SWF.

Esto permite que se puedan integrar fácilmente en sitios web sin necesidad de plugins.

- Ejemplo de Integración:

```
<canvas id="animationCanvas" width="800" height="600"></canvas>  
<script src="libs/createjs.min.js"></script>  
<script src="animation.js"></script>
```

- animation.js es el código generado automáticamente que maneja los movimientos y elementos animados.
- Se usa en banners, animaciones para redes sociales y juegos web sin necesidad de Flash.

2. Banners de Publicidad con Google Web Designer

Google Web Designer es una herramienta de Google para crear banners animados en <canvas>. Este tipo de banners se usan en plataformas como Google Ads, DoubleClick y AdSense.

- Ejemplo de un banner animado exportado desde Google Web Designer:

```
<div id="ad-container">  
  <canvas id="bannerCanvas" width="300" height="250"></canvas>  
</div>  
<script src="banner_animation.js"></script>
```

- Google Web Designer genera automáticamente banner_animation.js con las animaciones del banner.
 - Soporta interactividad, enlaces y eventos de clic.
 - Se usa en publicidad digital responsive.
- Ejemplo de Banners con Web Designer:
 - Ver Ejemplo de Google Web Designer

3. Visualización de Datos con Chart.js

Otro uso muy potente de <canvas> es en gráficos interactivos con librerías como Chart.js. Se usa en dashboards, analítica de datos, informes financieros y estadísticas en tiempo real.

- Ejemplo de un gráfico de barras con Chart.js:

```
<canvas id="myChart"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script>
  var ctx = document.getElementById('myChart').getContext('2d');
  var myChart = new Chart(ctx, {
    type: 'bar',
    data: {
      labels: ['Enero', 'Febrero', 'Marzo', 'Abril'],
      datasets: [{
        label: 'Ventas',
        data: [12, 19, 3, 5],
        backgroundColor: ['red', 'blue', 'green', 'orange']
      }]
    }
  });
</script>
```

- Características:
 - Gráficos responsivos y animados.
 - Diferentes tipos: barras, líneas, circulares, radar, etc.
 - Fácil de integrar con bases de datos.

4. Conclusión

El <canvas> ya no es solo para dibujar círculos y cuadrados. Hoy en día se usa en:

- Juegos y animaciones con Adobe Animate.
- Banners publicitarios animados con Google Web Designer.
- Gráficos interactivos con Chart.js.

Estas aplicaciones reemplazan tecnologías antiguas como Flash y permiten crear contenido multimedia más ligero y compatible con todos los navegadores.

Buenas Prácticas en el Uso de Animaciones

- No abusar de las animaciones.
- Usar transiciones suaves para mejorar la experiencia.
- Evitar animaciones que puedan afectar la accesibilidad.
- Optimizar el rendimiento con `will-change` y `transform` en lugar de `left/top`.

Elementos Interactivos

Creación de Elementos Interactivos

Ejemplo de botón con animación interactiva:

```
<button class="interactive-btn">Haz clic</button>
<style>
  .interactive-btn {
    background: #007bff;
    color: white;
    padding: 10px;
    transition: transform 0.2s;
  }

  .interactive-btn:active {
    transform: scale(0.9);
  }
</style>
```

Mapas Interactivos

Ejemplo con Google Maps API:

```
<div id="map" style="width:100%; height:400px;"></div>

<script>
  function initMap() {
    var location = { lat: 40.7128, lng: -74.0060 };
    var map = new google.maps.Map(document.getElementById("map"), {
      zoom: 10,
      center: location
    });
  }
</script>

<script async defer
src="https://maps.googleapis.com/maps/api/js?
key=AIzaSyC3FVJx3m79dCARkQfg42HQQiyTkYybXM&callback=initMap">
</script>
```

Ámbitos de Uso

- Webs interactivas y multimedia.
- Juegos web con HTML5 y JavaScript.
- Publicidad interactiva y banners animados.
- Aplicaciones web con experiencias dinámicas.

Ejercicio Final

Crear una landing page con una animación de entrada y un mapa interactivo mostrando la ubicación de un negocio.